

Overview of Gradient boosting

Subject: Gradient boosting

1.

for $m \geq 1$, where $h_m \in \mathcal{H}$ is a base learner function. Unfortunately, choosing the best function h_m at each step for an arbitrary loss function L is a computationally infeasible optimization problem in general. Therefore, we restrict our approach to a simplified version of the problem. The idea is to apply a steepest descent step to this minimization problem (functional gradient descent). The basic idea is to find a local minimum of the loss function by iterating on $F_{m-1}(x)$. In fact, the local maximum-descent direction of the loss function is the negative gradient. Hence, moving a small amount γ such that the linear approximation remains valid:

2.

Algorithm Many supervised learning problems involve an output variable y and a vector of input variables x , related to each other with some probabilistic distribution. The goal is to find some function $\hat{F}(x)$ that best approximates the output variable from the values of input variables. This is formalized by introducing some loss function $L(y, F(x))$ and minimizing it in expectation:

3.

$F_m(x) = F_{m-1}(x) - \gamma \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i))$

4.

Number of observations in leaves Gradient tree boosting implementations often also use regularization by limiting the minimum number of observations in trees' terminal nodes. It is used in the tree building process by ignoring any splits that lead to nodes containing fewer than this number of training set instances. Imposing this limit helps to reduce variance in predictions at leaves.

5.

$\gamma_m = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, F_m(x_i)) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) - \gamma \nabla F_{m-1} L(y_i, F_{m-1}(x_i)))$

6.

$\gamma_m = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) - \gamma \nabla F_{m-1} L(y_i, F_{m-1}(x_i)))$

7.

In the discrete case however, i.e. when the set H is finite, we choose the candidate function h closest to the gradient of L for which the coefficient γ may then be calculated with the aid of line search on the above equations. Note that this approach is a heuristic and therefore doesn't yield an exact solution to the given problem, but rather an approximation. In pseudocode, the generic gradient boosting method is:

8.

If we considered the continuous case, i.e., where H is the set of arbitrary differentiable functions on $\mathbb{R}$, we would update the model in accordance with the following equations

9.

Gradient tree boosting Gradient boosting is typically used with decision trees (especially CARTs) of a fixed size as base learners. For this special case, Friedman proposes a modification to gradient boosting method which improves the quality of fit of each base learner. Generic gradient boosting at the m -th step would fit a decision tree $h_m(x)$ to pseudo-residuals. Let J_m be the number of its leaves. The tree partitions the input space into J_m disjoint regions $R_{1m}, \dots, R_{J_m m}$ and predicts a constant value in each region. Using the indicator notation, the output of $h_m(x)$ for input x can be written as the sum:

10.

where γ_m is the weight at stage m . We are usually given a training set $\{(x_1, y_1), \dots, (x_n, y_n)\}$ of known values of x and corresponding values of y . In accordance with the empirical risk minimization principle, the method tries to find an approximation $\hat{F}(x)$ that minimizes the average value of the loss function on the training set, i.e., minimizes the empirical risk. It does so by starting with a model, consisting of a constant function $F_0(x)$, and incrementally expands it in a greedy fashion:

11.

When the loss $L(\cdot, \cdot)$ is mean-squared error (MSE) the coefficients γ_j coincide with the coefficients of the tree-fitting procedure b_{jm} .

12.

where $\gamma > 0$. For small γ , this implies that $L(y_i, F_m(x_i)) \leq L(y_i, F_{m-1}(x_i))$.

13.

where b_{jm} is the value predicted in the region R_{jm} . Then the coefficients b_{jm} are multiplied by some value γ_m

γ_m , chosen using line search so as to minimize the loss function, and the model is updated as follows:

14.

Informal introduction (This section follows the exposition by Cheng Li.) Like other boosting methods, gradient boosting combines weak "learners" into a single strong learner iteratively. It is easiest to explain in the least-squares regression setting, where the goal is to teach a model F to predict values of the form $\hat{y} = F(x)$ by minimizing the mean squared error $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$, where i indexes over some training set of size n of actual values of the output variable y :